Lab 6.2. Integrating Argo Events with External Systems

This lab is designed to expand your experience with Argo Events by integrating it with Apache Pulsar. After having previously triggered Argo Events workflows using a simple HTTP request, you will now use Pulsar, a distributed messaging system, as a different event source.

Objectives

- Gain practical experience in setting up and triggering workflows in Argo Events using messages from external systems.

Prerequisites

- A Kubernetes cluster is available with the components installed from Lab 6.1.
- kubectl is installed and configured to communicate with your Kubernetes cluster.
- Internet access for downloading necessary files and packages.

Use Apache Pulsar with Argo Events

1. Triggering a Workflow with Pulsar

Deploy Apache Pulsar in your cluster with the following command:

```
kubectl -n argo-events apply -f
https://raw.githubusercontent.com/lftraining/LFS256-code/main/argoevents/pulsar.yaml
```

Check the status of the Pulsar pod and note the name. Run the command below:

```
kubectl get pods -n argo-events
```

Next, we need to port forward the Pulsar pod to enable direct communication between your local machine and the Pulsar service running in the Kubernetes cluster. This step is crucial for Argo Events to interact with Pulsar for triggering workflows. We do that with the following command:

```
kubectl -n argo-events port-forward <POD NAME OF PULSAR> 6650:6650
```

Replace **POD NAME OF PULSAR** with the actual name of the Pulsar pod obtained from the **kubectl get pods** command.

Set up the event source for Argo Events to listen to Pulsar messages. This configures Argo Events to connect and listen to Pulsar. Run the command below:

```
kubectl -n argo-events apply -f
https://raw.githubusercontent.com/argoproj/argo-events/stable/examples
/event-sources/pulsar.yaml
```

Deploy the sensor for reacting to Pulsar events as defined by the YAML below:

```
cat <<EOF | kubectl -n argo-events apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Sensor
metadata:
  name: pulsar
spec:
  template:
    serviceAccountName: operate-workflow-sa
  dependencies:
    - name: test-dep
      eventSourceName: pulsar
      eventName: example
  triggers:
    - template:
        name: workflow-trigger
        k8s:
          operation: create
          source:
            resource:
              apiVersion: argoproj.io/v1alpha1
              kind: Workflow
              metadata:
                generateName: pulsar-wf-
              spec:
                entrypoint: cowsay
                arguments:
```

```
                parameters:
                  - name: message
                    # value will get overridden by the event payload
                    value: hello world
              templates:
                - name: cowsay
                  inputs:
                    parameters:
                      - name: message
                  container:
                    image: rancher/cowsay:latest
                    command: [cowsay]
                    args: ["{{inputs.parameters.message}}"]
        parameters:
          - src:
              dependencyName: test-dep
              dataKey: body
            dest: spec.arguments.parameters.0.value
EOF
```

This sets up the actions to be taken in response to events detected by Argo Events:

Now, everything is set up to trigger the event. To interact with the Pulsar pod, use the following commands:

```
kubectl -n argo-events get pods
```

```
kubectl -n argo-events exec -it <NAME OF PULSAR POD> -- /bin/bash
```

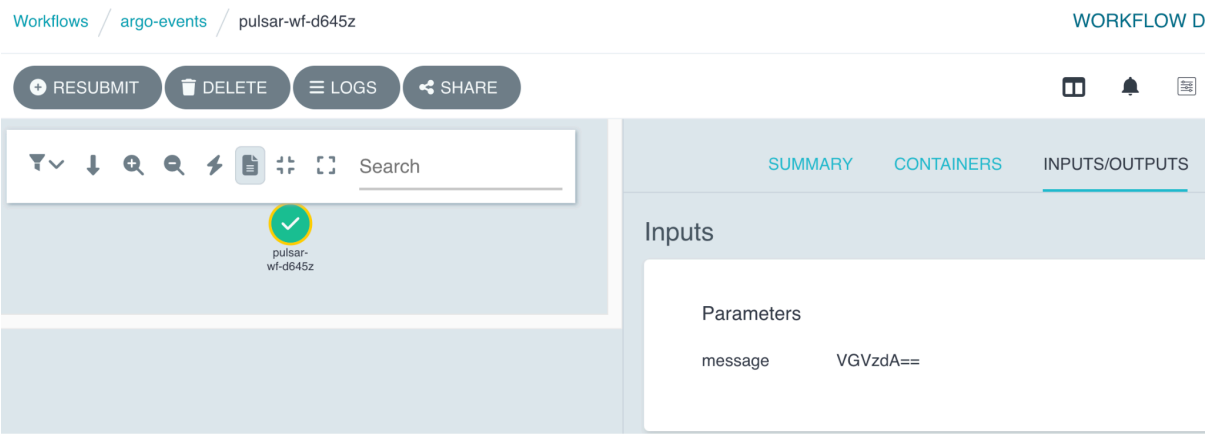
Inside the Pulsar pod, navigate to the `bin` directory and send a test message:

```
cd bin
```

```
./pulsar-client produce test --messages "Test"
```

2. Inspecting the Triggered Workflow

After sending the "Test" message via Pulsar, it triggers an Argo workflow. In the Argo UI, you can see the message in the workflow the same as in the first lab.



Message of Pulsar shown in the workflow

You can also see the output from the pod in the cluster too.

First, find the **`workflow`** pod (it should have `**`-wf`**` in the suffix. Run the following command:

```
$ kubectl get pods -n argo-events | grep pulsar
```

Output:

```
pulsar-758c47fc7-mpgd7                       1/1     Running     0
5m52s

pulsar-eventsource-ql95p-77b5bb695b-wv9q6    1/1     Running     0
4m39s

pulsar-sensor-rpqzd-df45d7dbc-drx7f          1/1     Running     0
4m17s

pulsar-wf-9lxlv                              0/2     Completed   0
3m10s
```

Then check the logs with the following command:

```
$ kubectl logs -n argo-events pulsar-wf-9lxlv
```

Output:

```
 ____________
< VGVzdA== >
 ----------
        \   ^__^
```

```
         \  (oo)\_______
            (__)\       )\/\
                ||----w |
                ||     ||
```

```
time="2025-11-04T04:22:54.409Z" level=info msg="sub-process exited"
argo=true error="<nil>"
```

However, this message is encoded in Base64, a common practice in Apache Pulsar for efficient and reliable data serialization and transmission. To read the message correctly, you'll need to decode it from Base64. Run the command below:

```
$ echo "VGVzdA==" | base64 -d
```

Output:

```
Test
```

In this lab, we focused on the integration of Argo Events with Apache Pulsar, demonstrating how Argo Events can be configured to interact with external messaging systems. This exercise highlights Argo Events' versatility in integrating with different external systems, enhancing its capability to manage complex, event-driven architectures in cloud-native environments.